

Plant Load Radar: Slide Script

Chin Hin pre-interview · about 12 minutes · read aloud, then make it your own words

Delivery tips: pause after every big number (RM82,000, 2,000 m³). Say "illustrative" once on slides 4 and 9. Look at the camera, not the slide. If short on time, cut slides 6 and 11 to one sentence each.

1. Plant Load Radar · 20 sec

Hi, I'm Azim, a final-year Computer Science student majoring in Data Analytics at UTP. I'm pitching Plant Load Radar for Chin Hin's Capacity and Demand Intelligence challenge. In one sentence: when a plant can't serve everyone, it should decide who goes first by what's actually at stake in ringgit, and it should do that every week, with a clear owner, in tools it already has. I'll spend one slide on the problem, then go straight into the solution.

2. Two fair priorities, and no shared rule · 60 sec

Here's the problem on one page. Chin Hin's plants make AAC blocks, precast and ready-mix for two customers: outside contractors, and Chin Hin's own projects, which are growing fast. Property revenue grew 31 percent and construction 22 percent in the first half of this financial year. When capacity is tight, the plant naturally chases volume and margin, while the group wants its own projects on schedule. Both are reasonable, but there's no shared rule, so it's decided deal by deal. On top of that, each plant forecasts on its own in spreadsheets, so there's excess stock in one place and missed orders in another. And the brief rules out the easy answers: no new ERP, no consultant, no transfer-pricing policy, and no escalating it upwards.

3. Plant Load Radar: one rule, one owner, one board · 45 sec

So here's my answer: Plant Load Radar. It's three things. One, a rule: when capacity is short, serve whichever request has more ringgit at stake, the internal project's cost of delay or the outside order's margin. Two, an owner: the plant scheduler applies that rule every week, and anything the rule can't settle goes to a 30-minute huddle, not up the chain. Three, a board: one view of all committed demand against capacity, built in Excel and Power Apps, which the plant can actually run. Underneath, an agentic AI layer does the legwork: it reads messy WhatsApp and paper orders, tests the options against capacity, and drafts the messages. But the maths comes from a calculator, and people make the call. Let me take each part in turn.

4. More ringgit at stake wins · 90 sec

Part one, the rule. When capacity is short, the request with more ringgit at stake goes first. Here's an example. Week 43 is 2,000 cubic metres short. Project A is one of Chin Hin's own developments. If its handover slips past the legal deadline, the Housing Development Act makes the developer pay buyers 10 percent a year of the purchase price, day by day. For a 300 million ringgit block, that's about 82,000 ringgit a day. Contractor B's order is worth about 18,000 ringgit in margin. So Project A goes first, and Contractor B is offered the next free week instead of being turned away. To keep it fair, there are four rules of thumb. Projects that confirm call-offs early get protected capacity, but unused slots are released. Confirmed orders are never bumped, so the rule only decides new requests. If it's a close call, the earlier confirmed request wins. And because AAC can be stocked, quiet weeks pre-build for busy ones, within a stock limit so we don't tie up cash. These numbers are illustrative; the real ones come from Chin Hin's data.

5. Who decides, and when · 60 sec

Part two, the owner. A rule only works if someone clearly applies it. Project planners and sales confirm orders and call-offs as early as they can, because early confirmation earns protected capacity. The plant scheduler updates the board every week and applies the rule. If something genuinely doesn't fit, the plant manager and the project director settle it in a fixed 30-minute weekly huddle, using the same ringgit comparison. And group finance reviews the ringgit saved and lost once a month. The key point: nothing goes up the chain. The brief rules out escalation, and this design doesn't need it, because the plant manager owns every decision made within the rule.

6. How Plant Load Radar works, every week · 60 sec

Part three, the board, and how the whole thing runs in a normal week. One: orders arrive the way they do today, by WhatsApp, on paper delivery orders, or in Excel. Two: an intake agent reads each one and turns it into a clean order line, and a person confirms anything it isn't sure about. Three: everything lands on one board, capacity against firm, likely and possible demand, week by week, internal and external together. Four: a planner agent finds the short weeks and tests the options, serve, move to another week, or pre-build stock, using the rule calculator. Five: the scheduler approves, and generative AI drafts the replies to customers and the ledger note. It all runs on Excel or SharePoint with a Power Apps front end, so there's no new ERP. Agents do the legwork, a calculator does the maths, and people make the call. Next, the AI layer itself.

7. Four agents, one human checkpoint · 75 sec

Here's the AI layer itself, and this is where my background comes in. It's an agentic pipeline of four agents with one human checkpoint. Agent one, intake, reads WhatsApp messages, photos of paper delivery orders and Excel files, and turns them into structured order lines. Agent two, matching, links each order to the right project or customer, removes duplicates, and tags it firm, likely or possible. Agent three, the planner, finds short weeks and tests the options, serve, move to another week, or pre-build stock, by calling the capacity check and the rule calculator as tools. Agent four, the writer, uses generative AI to draft the huddle brief, the customer replies and the ledger notes. Then the scheduler approves; nothing is sent or committed without a person. Three guardrails make it trustworthy: the numbers always come from tools, never from the model; every field shows its source and how confident the agent is; and it's orchestrated with LangGraph or Microsoft Foundry, triggered from Power Automate, so it fits Chin Hin's Microsoft stack. What's still premature is forecasting models and letting AI decide; those come later, once the data is clean.

8. From a WhatsApp message to a clean order · 45 sec

Here's the intake agent up close, because this is where AI genuinely earns its place first. On the left is what an order really looks like at a plant: a WhatsApp message, short, informal, half in shorthand. On the right is what the board receives: customer, product, volume, date, each marked confirmed. The last field is the interesting one. The message says "can confirm" but there's no formal call-off yet, so the AI marks it as likely, not firm, and flags it for a person to check. That's the design principle: if the AI isn't sure, it asks; it never guesses. It's the same source-first approach I used for VERA, the knowledge assistant I built during my PETRONAS internship.

9. The dashboard: Plant Load Radar in one screen · 60 sec

This is what the scheduler would actually use: Plant Load Radar in one screen. On the left, simple navigation: the board, the orders inbox, decisions, the ledger, and "ask the board". At the top, four numbers: two weeks are short, three new orders need checking, one decision is waiting, and the plant has protected about 246,000 ringgit this month. The chart shows spare or short capacity for the next six weeks, so you can see at a glance that week 43 is 2,000 cubic metres short and week 44 has 4,100 spare. The red card on the right is the planner agent's recommendation, with the ringgit on both sides, and the scheduler just taps approve or change. At the bottom is the orders inbox: everything the intake agent read this morning, from WhatsApp and paper, with anything uncertain marked for checking. It's built in Power Apps on top of Excel or SharePoint, so the plant doesn't need a new system. All numbers are illustrative.

10. Generative AI does the writing, people send it · 45 sec

This is where generative AI earns its keep: the writing that eats a scheduler's time. On the left, the writer agent drafts the weekly huddle brief for the plant manager: what's short, what's recommended and why in ringgit, and the one decision that actually needs his call. On the right, it drafts the WhatsApp reply to Contractor B, offering the next free week instead of a flat no. It waits for approval; the scheduler taps approve or edits it first. And at the bottom, the scheduler can simply ask the board a question in plain language, like whether we can take 800 cubic metres for Johor next week, and the agent answers by running the calculator, not by guessing. All the numbers come from the rule calculator; the model only writes the words.

11. Rolling it out: about three months at one plant · 45 sec

Here's how we'd roll it out, at one plant first, over about three months. Month one is understand and build: sit with the scheduler, rebuild recent decisions from WhatsApp and paper so we know what today actually costs, and build the dashboard and the agents. By the end of month one, the plant has a working dashboard with its real orders. Month two is agree and go live: agree the rule with the plant manager and a project director, start the weekly 30-minute huddle, and log every decision with its ringgit. By the end, the plant is deciding by the rule every week. Month three is measure and decide: compare cash, margin and delays with before, fix what didn't work, and decide whether to roll it out to precast and ready-mix. We'll know it worked if we see cash released from excess stock, outside margin no longer lost, and fewer project delay days.

12. About me · 75 sec

A bit about me. On achievements: I've been on the Dean's List three times, I have an IEEE paper accepted at MCAIT 2026, which I'll present in Madinah, my team EduNova qualified for the HACKaSTONE 2026 Grand Final in Amsterdam, and we presented Synapse at Codex Community Day. Outside tech, I led UTP's esports club, and our team won gold twice and went on to represent Malaysia in Singapore. On projects: I built VERA, a RAG knowledge assistant, during my PETRONAS internship; I'm the technical lead for system automations at UTP's Career Development Office, where I rebuilt 14 undocumented Power Automate flows; my final-year project Arcana is a multi-agent pipeline on LangGraph, the same pattern as the agent layer I'm proposing; and I founded PocketLab to take Synapse further. I've also led and organised: the esports club, the LEADX symposium sponsorship committee, GDSC, and an outreach programme in Seoul. The common thread: I build agentic and generative AI that people actually use, and I can show the working.

13. Thank you · 15 sec

To sum up: one rule, one owner, one board, with agents doing the legwork and people making the call. Thank you for your time, and I'm happy to take any questions.

